MIS41110 Practical Sheet, Week 3

Covers the Week 3 lecture (flow control, UML activity diagrams, pseudo-code) · **Released** with the Week 3 slides · **Worked** Monday of Week 4 · **Solutions** that afternoon

Solutions from your tutors after the day's last session. Still no lists or dicts; they arrive Friday.

Exercise 1: draw before you code

Draw the **UML activity diagram** for this ration policy (paper or any drawing tool; the official notation from the lecture: start/end nodes, action boxes, decision diamonds, labeled edges):

```
crew = int(input("Crew on station: "))
while crew != 0:
    if crew > 40:
        boxes = 3
    elif crew > 20:
        boxes = 2
    else:
        boxes = 1
    print(f"Open {boxes} ration boxes.")
    crew = int(input("Crew on station: "))
```

(a) In one sentence: what does the diagram show that the code hides?

Exercise 2: the tag validator

Penguin tags are numbered `1` to `500`. Write a program that keeps asking `Tag number:` **until** the answer is in range, then confirms:

```
Tag number: 0
Tags run 1-500. Try again.
Tag number: 612
Tags run 1-500. Try again.
Tag number: 77
Tag 77 logged.
```

(a) Which loop fits and why: `while` or `for`?

Exercise 3: the comms menu

Every station program you will ever meet runs a menu loop. Build one:

```
[1] Weather  [2] Colony count  [0] Sign off
Choose: 1
Outside: -17.1 C, wind 35 km/h
[1] Weather  [2] Colony count  [0] Sign off
Choose: 5
Please choose 1, 2 or 0.
[1] Weather  [2] Colony count  [0] Sign off
Choose: 2
North Ridge: 242 adelie penguins at last count.
[1] Weather  [2] Colony count  [0] Sign off
Choose: 0
Farpoint signing off.
```

(Hard-code the weather line and the count; the loop is the exercise.)

Exercise 4: `for` and `range()` drills

1. Print the supply-drop countdown `10 9 8 ... 1 LIFT` (a `for` over a `range` with a negative step, then one final print).
2. Print a divider of 30 dashes using a loop (yes, `"-" * 30` is better; the loop is the point today).
3. Loop over the string `"Farpoint"` and count its lowercase vowels (`if letter in "aeiou"`), then print the count.

Exercise 5: blizzard watch

`data/field_notes.txt` holds one field note per line (first line is a header). The `data/` folder comes from Brightspace with this sheet: put it beside your program and run from that folder, or Python cannot find the file. Using `for line in open(...)`, count how many notes mention `blizzard`, and print:

```
3 blizzard notes this season.
```

(a) Your loop reads the header line too. Does that change this count? When *would* it?

Exercise 6 ☆: one colony's file (if the session runs short, finish at home)

Write `colony_extract.py`. It takes **two arguments**: the notes file and a colony name. It writes every note line that starts with that colony into a new file named `<colony>.txt`, and reports. With the wrong number of arguments it prints a usage message and stops (wrap the work in `if`/`else`; no exceptions needed yet). Two new tools, take them as given today: `len(sys.argv)` counts the arguments (three when two are passed: the program's own name is item 0), and `line.startswith(colony + ",")` answers "does this line start with that?" (a string method, the same dot-call family as `.strip()`):

```
$ python3 colony_extract.py
USAGE: colony_extract.py notes_file colony_name
$ python3 colony_extract.py data/field_notes.txt Windward
6 notes written to Windward.txt.
$ python3 colony_extract.py data/field_notes.txt "North Ridge"
7 notes written to North Ridge.txt.
```

Two of the three colonies are two words, so the last run quotes the name. Unquoted, `North` and `Ridge` arrive as two arguments, and your program, rightly, prints its usage message.

Meaningful names, comments where needed, and test every branch: the happy path, the wrong path, and the boundary.